Solveur de mécanique des structures

Max Billette

Mars 2026

1 Introduction **2**

2 Rappel de mécanique des structures **2**
2.1 Définitions 2
2.2 Équilibre dans un nœud 3
2.3 Résolution du système 4
2.4 Déformations 5
2.5 Exemple 6
2.6 Hypothèses et limites de validité 8

3 Construction du code **8**
3.1 Architecture du code 8
3.2 Matrice d'équilibre 10
3.2.1 Construction 10
3.2.2 Inversion et singularité 11
3.3 Entrée et sorties 12
3.3.1 Entrée 12
3.3.2 Sorties 13

4 Conclusion **15**

Annexes **16**

A Pseudo-code de la fonction `_constru_matrice` **16**

B Pseudo-code de la fonction `_inversion_matrice` **17**

C Exemple de texte d'entrée du code **19**

Références **20**

1 Introduction

Ce projet vise à coder un solveur simple de forces et déplacements dans un treillis isostatique. Le programme sera écrit en Python. Le code source est disponible sur GitHub [1] avec trois exemples. L'objectif de ce projet n'est pas de coder un solveur particulièrement performant mais plutôt d'explorer le fonctionnement des outils de l'ingénieur. On considérera des treillis isostatiques subissant de petites déformations avec des barres ayant un comportement élastique linéaire reliées par des jonctions parfaites. Après avoir fait des rappels sur la théorie de la mécanique des structures, on s'intéressera à la construction du code : son architecture, le solveur en lui même, les résultats obtenus et les mécanismes d'entrées et de sorties du code. On conclura sur les limites de ce solveur et sur les potentielles améliorations.

2 Rappel de mécanique des structures

On rappelle dans cette section les bases de la théorie de la mécanique des structures, afin de connaître les équations et hypothèses nécessaires à la construction du solveur.

2.1 Définitions

La notion de treillis est définie comme suit par la référence [2] :

> Un treillis [ ] est un assemblage de poutres reliées entre elles par des *articulations rotoïdes* parfaites, appelées nœuds du treillis. Le chargement, constitué par des forces ponctuelles, est appliqué seulement aux *nœuds* du treillis. Dans ces conditions, chaque poutre ne travaille qu'en traction-compression. De telles poutres sont appelées des barres et il y a transmission des efforts dans la direction des *barres* uniquement.

On note que les systèmes résolus ici sont en deux dimensions. Dans ce cas, les articulations rotoïdes se réduisent à des articulations *pivot*. Par ailleurs, ce solveur ne peut calculer les forces et déplacements que dans un treillis isostatique. La notion d'isostatique est définie comme suit par la référence [3] :

> Un système est dit isostatique si toutes les réactions d'appuis ainsi que les contraintes généralisées en tout point de la structure peuvent être déterminées par la seule utilisation des équations d'équilibre.

Dans le système, les positions des nœuds et des barres, les attaches des nœuds et les forces en présence sont connus. On cherche à connaître les tensions dans les barres, les réactions aux supports et les déplacements des nœuds. L'extension aux systèmes hyperstatiques est discutée en conclusion.

2.2 Équilibre dans un nœud

Le système étant statique, la somme des forces appliquées à chaque nœud est nulle. Considérons un nœud quelconque k, soumis à une force extérieure $\vec{F}_k$ et connecté à b barres. Chaque barre i est caractérisée par sa direction unitaire $\vec{u}_i$ et son effort normal N_i. En posant l'équilibre pour ce nœud k on obtient cette équation :

$$\vec{F} + \sum_{i=1}^{b} N_i \vec{u}_i = \vec{0} \tag{1}$$

Puisque le problème est en deux dimensions, on a deux équations par nœud

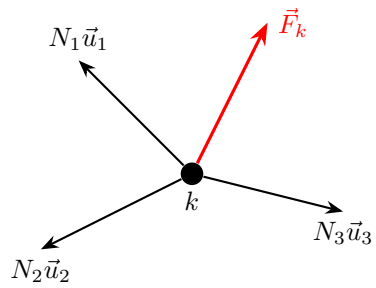


FIGURE 1 – Équilibre du nœud k

qu'on obtient en projetant cette équation sur $\vec{u}_x$ et sur $\vec{u}_y$. Chaque barre i connectant un nœud j à un nœud k est caractérisée par son angle $\theta_{j,k}$ avec l'axe $\vec{u}_x$, défini dans le sens de j vers k :

$$\theta_{j,k} = \tan^{-1}\left(\frac{y_k - y_j}{x_k - x_j}\right) \tag{2}$$

On note que $\theta_{j,k} = \theta_{k,j} + \pi$, ce qui implique $\cos(\theta_{j,k}) = -\cos(\theta_{k,j})$ et $\sin(\theta_{j,k}) = -\sin(\theta_{k,j})$. En notant ici θ_i l'angle de la barre i orienté du nœud concerné par l'équation au nœud à l'extrémité de la barre, on a alors :

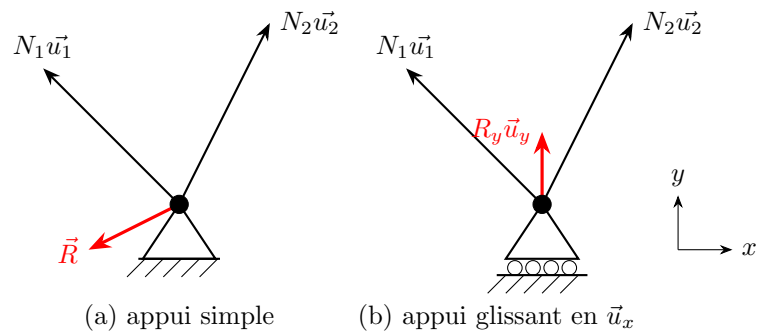
$$F_x + \sum_{i=1}^{b} N_i \ \cos(\theta_i) = 0 \tag{3}$$

$$F_y + \sum_{i=1}^{b} N_i \ \sin(\theta_i) = 0 \tag{4}$$

Dans le cas d'un nœud avec attache (fig. 2), on ajoute une force $\vec{R}$ représentant la réaction de l'appui :

$$\vec{F} + \sum_{i=1}^{b} N_i \vec{u}_i + \vec{R} = \vec{0} \tag{5}$$

On peut écrire la réaction $\vec{R} = R_x\vec{u}_x + R_y\vec{u}_y$. Si le nœud est glissant en $\vec{u}_x$, alors la force de réaction est colinéaire à $\vec{u}_y$: $R_x = 0$. Si le nœud est glissant en $\vec{u}_y$, alors la force de réaction est colinéaire à $\vec{u}_x$: $R_y = 0$.



(a) appui simple (b) appui glissant en $\vec{u}_x$

FIGURE 2 – Réactions aux supports

2.3 Résolution du système

En écrivant ces équations pour chaque nœud on obtient d'une part $2 \times n$ équations, avec n le nombre de nœuds. D'autre part, si on note b le nombre de barres et a le nombre d'action de liaison on a $a + b$ inconnues dans le système, les inconnues étant les tensions dans les barres N_i et les réactions aux support $R_{k,x}$ et $R_{k,y}$. On calcule a comme suit :

$$a = \sum_{i=1}^{n} a_i \quad \text{avec} \quad \begin{cases} a_i & = 0 \quad \text{pour un nœud sans attache} \\ a_i & = 1 \quad \text{pour un nœud glissant en } \vec{u}_x \text{ ou } \vec{u}_y \\ a_i & = 2 \quad \text{pour un nœud fixe} \end{cases} \tag{6}$$

Puisque les inconnues de toutes ces équations sont aux premiers degrés, on a un système linéaire constitué de $2 \times n$ équations avec $a + b$ inconnues. Comme précisé précédemment, on ne considère que les systèmes isostatiques. En d'autres termes, on a une égalité entre le nombre d'équation et le nombre d'inconnues du système :

$$2 \times n = a + b \tag{7}$$

Avec toutes ces informations, on peut maintenant construire une matrice carrée d'ordre $2 \times n$ qu'on appellera matrice d'équilibre. Cette matrice permet d'encoder les relations géométriques du système de manière compact afin de pouvoir le résoudre numériquement ensuite. Si on note $\mathbf{K}$ cette matrice d'équilibre, $\vec{S}$ le vecteur des inconnues et $\vec{F}$ le vecteur des forces appliquées à chaque nœud, on peut écrire le système d'équations dans la forme suivante :

$$\vec{F} + \mathbf{K} \times \vec{S} = \vec{0} \tag{8}$$

Le système étant isostatique et donc la matrice carrée, et dans le cas où la matrice est inversible, on peut manipuler l'équation (8) pour la réécrire comme suit :

$$\vec{S} = -\mathbf{K^{-1}} \times \vec{F} \tag{9}$$

Cette manipulation n'est possible que si la matrice d'équilibre est inversible. Si on obtient une matrice singulière (non-inversible), l'équation (8) possède une infinité de solutions. Dans ce cas, le système étudié est un mécanisme.

La référence [4] offre un cours complet sur les systèmes linéaires et leurs résolutions.

2.4 Déformations

En reprenant le principe de Saint-Venant pour une poutre en tension N uniaxiale, on peut avoir le tenseur des contraintes dans la barre, qui est constitué d'un seul terme non nul dans la direction $\vec{u_l}$ de la barre égale à :

$$\sigma_{l,l} = \frac{N}{A} \tag{10}$$

Avec A l'aire de la section. On utilise la loi de Hooke pour connaître la déformation axiale de la barre :

$$\epsilon = \frac{\sigma_{l,l}}{E} = \frac{N}{EA} \tag{11}$$

Avec E le module d'Young et EA la rigidité axiale de la barre. Si on note l la longueur de la barre, et δl son élongation, on peut exprimer sa loi de comportement élastique :

$$\frac{\delta l}{l} = \epsilon = \frac{N}{EA} \tag{12}$$

Pour calculer le déplacement de chaque nœud, on utilise la méthode de Mohr. Cette méthode égalise un travail de force extérieure à une énergie de déformation. Pour chaque nœud k, on va appliquer une force virtuelle $\bar{F}_k$. Cette force va induire un déplacement du nœud qu'on note Δ_k et qui est dans la direction de la force $\bar{F}_k$. Le travail W_{ext} associé cette force est :

$$W_{k,ext} = |\bar{F}_k|\Delta_k \tag{13}$$

D'autre part, l'énergie de déformation W_{int} dans le treillis se calcule comme suit :

$$W_{int} = \sum_{i=1}^{b} n_i \delta_i = \sum_{i=1}^{b} \frac{\bar{n}_i N_i l_i}{E_i A_i} \tag{14}$$

Avec $\bar{n}_i$ la tension induite par la force virtuelle $\bar{F}_k$ dans la barre i. On peut égaliser l'énergie de déformation avec le travail de la force virtuelle pour isoler le déplacement du nœud k dans la direction de $\bar{F}_k$, on obtient :

$$\Delta_k = \sum_{i=1}^{b} \bar{n}_i \delta_i = \frac{1}{|\bar{F}_k|} \sum_{i=1}^{b} \frac{\bar{n}_i N_i l_i}{E_i A_i} \tag{15}$$

On peut maintenant calculer ce déplacement pour chaque nœud, en prenant soin de choisir une force virtuelle en $\vec{u}_x$ puis en $\vec{u}_y$, pour obtenir un déplacement $\Delta_{k,x}$ et $\Delta_{k,y}$.

2.5 Exemple

On prend un exemple très simple pour illustrer les équations ci-dessus : deux nœuds reliés par une barre horizontale de longueur $l = 1m$ et de rigidité axiale $EA = 1,5kN$, un nœud (1) glissant en $\vec{u}_x$ et un nœud (2) fixe (fig. 3). Le nœud 1 subit une force $\vec{F_1} = F_{1,x}\vec{u}_x + F_{1,y}\vec{u}_y$ avec $F_{1,x} = -25N$ et $F_{1,y} = 10N$. Cet exemple est disponible sur GitHub en tant qu'*exemple0.txt*.

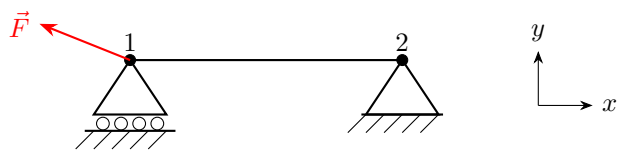


FIGURE 3 – Treillis d'une barre, un appui glissant et un appui fixe

On a ici deux nœuds, donc $2 \times n = 4$ équations. On a aussi une barre ($b = 1$) et trois actions de liaisons ($a = 3$, une sur le nœud 1 glissant en $\vec{u}_x$ et deux sur le nœud 2 fixe) donc $a + b = 4$. La condition (7) est bien vérifié, le système est isostatique. On écrit les équations d'équilibre pour les deux nœuds :

$$\begin{array}{ll} \text{Nœud 1} & \left\{ \begin{array}{lllllll} F_{1,x} & + & N_{1,2} & \cos(\theta_{1,2}) & + & R_{1,x} & = 0 \\ F_{1,y} & + & N_{1,2} & \sin(\theta_{1,2}) & + & R_{1,y} & = 0 \end{array} \right. \\ \text{Nœud 2} & \left\{ \begin{array}{lllllll} & & N_{1,2} & \cos(\theta_{2,1}) & + & R_{2,x} & = 0 \\ & & N_{1,2} & \sin(\theta_{2,1}) & + & R_{2,y} & = 0 \end{array} \right. \end{array} \tag{16}$$

La barre étant horizontale, on a $\sin(\theta_{1,2})$ et $\sin(\theta_{2,1})$ nuls, $\cos(\theta_{1,2}) = 1$ et $\cos(\theta_{2,1}) = -1$. De plus le nœud 1 est glissant en $\vec{u}_x$, donc $R_{1,x} = 0$. On réécrit alors le système d'équation :

$$\begin{array}{ll} \text{Nœud 1} & \left\{ \begin{array}{lllllll} F_{1,x} & + & N_{1,2} & + & & = 0 \\ F_{1,y} & + & & + & R_{1,y} & = 0 \end{array} \right. \\ \text{Nœud 2} & \left\{ \begin{array}{lllllll} & - & N_{1,2} & + & R_{2,x} & = 0 \\ & & & & R_{2,y} & = 0 \end{array} \right. \end{array} \tag{17}$$

On identifie le vecteur des forces $\vec{F}$, le vecteur des inconnues $\vec{S}$ et la matrice d'équilibre $\mathbf{K}$:

$$\vec{F} = \begin{pmatrix} F_{1,x} \\ F_{1,y} \\ 0 \\ 0 \end{pmatrix}, \quad \vec{S} = \begin{pmatrix} N_{1,2} \\ R_{1,y} \\ R_{2,x} \\ R_{2,y} \end{pmatrix} \quad \text{et} \quad \mathbf{K} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{18}$$

On note que l'ordre dans lequel les inconnues sont encodées n'importe pas tant que la résolution du système reste cohérente avec ce choix. L'inverse de la matrice est :

$$\mathbf{K}^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{19}$$

On peut donc obtenir le vecteur des inconnues par multiplication matricielle en reprenant l'équation (9) :

$$\vec{S} = -\mathbf{K^{-1}} \times \vec{F} = \begin{pmatrix} -F_{1,x} \\ -F_{1,y} \\ -F_{1,x} \\ 0 \end{pmatrix} \tag{20}$$

Au final on obtient pour valeur des inconnues :

$$\vec{S} = \begin{pmatrix} N_{1,2} & = 25N \\ R_{1,y} & = -10N \\ R_{2,x} & = 25N \\ R_{2,y} & = 0N \end{pmatrix} \tag{21}$$

Pour le calcul des déformations, on pourrait utiliser la loi de Hooke directement puisque le système est constitué d'une barre seulement. Cependant on utilise ici la méthode de Mohr pour illustrer la partie 2.4.

Pour connaître un déplacement d'un nœud k dans une direction i ($\vec{u}_x$ ou $\vec{u}_y$), on applique une force virtuelle $\bar{F}_{k,i}$ puis on calcule la tension virtuelle $\bar{n}_{1,2}$ induite dans la barre. Ce calcul est assez simple puisque la matrice d'équilibre inversée $\mathbf{K^{-1}}$ est déjà connu. On commence par le déplacement $\Delta_{1,x}$ du nœud 1 en $\vec{u}_x$. La force virtuelle $\bar{F}_{1,x}$ peut être choisie arbitrairement mais la valeur de $\bar{F}_{1,x} = 1N$ est choisi puisqu'elle permet de simplifier le terme $\frac{1}{|\bar{F}_k|}$ de l'équation (15). On obtient un vecteur de forces virtuelles :

$$\vec{\bar{F}} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \text{ et comme solution virtuelle : } \vec{\bar{S}} = -\mathbf{K^{-1}} \times \vec{\bar{F}} = \begin{pmatrix} -1 \\ 0 \\ -1 \\ 0 \end{pmatrix} \tag{22}$$

La tension virtuelle $\bar{n}_{1,2} = -1N$ permet de calculer enfin le déplacement avec l'équation (15) :

$$\Delta_{1,x} = \frac{1}{|\bar{F}_{1,x}|} \frac{\bar{n}_{1,2} N_{1,2} l}{EA} = l/60 = 1,67 \times 10^{-2} m \tag{23}$$

Le déplacement $\Delta_{1,x}$ du nœud 1 en $\vec{u}_x$ est donc de $1,67 \times 10^{-2} m$. Le nœud 2 étant fixe et le nœud 1 étant glissant en $\vec{u}_x$ (donc fixe en $\vec{u}_y$), le seul déplacement possible pour les nœuds du système est le déplacement $\Delta_{1,x}$. Le calcul des

déplacements $\Delta_{1,y}$, $\Delta_{2,x}$ et $\Delta_{2,y}$ n'est donc pas nécessaire, on sait d'emblée qu'ils sont nuls. On peut cependant s'en convaincre en utilisant la méthode de Mohr pour un de ces déplacements. Par exemple pour calculer $\Delta_{2,y}$ on applique une force virtuelle $\bar{F}_{2,y} = 1N$ sur le nœud 2 dans la direction $\vec{u}_y$. Cette force virtuelle donne une tension virtuelle $\bar{n}_{1,2} = 0$. Le déplacement $\Delta_{2,y}$ est donc de $0m$.

2.6 Hypothèses et limites de validité

Le solveur repose sur les hypothèses suivantes :

- **Petits déplacements :** les déplacements des nœuds sont supposés très petits devant les dimensions de la structure. La géométrie du treillis est donc considérée inchangée après déformation, ce qui justifie l'utilisation des positions initiales des nœuds pour construire la matrice d'équilibre.
- **Élasticité linéaire :** le comportement de chaque barre suit la loi de Hooke. En particulier, la rigidité axiale EA est supposée constante.
- **Articulations parfaites :** les nœuds sont supposés parfaitement articulés, donc aucun moment n'est transmis entre les barres.
- **Isostaticité :** le solveur ne traite que les systèmes vérifiant $2n = a + b$. Les systèmes hyperstatiques ou hypostatiques ne sont pas pris en charge.

3 Construction du code

Cette section donne les grandes étapes de la construction du code. Le code est écrit en python sur l'IDE *Visual Studio Code*. On verra d'abord l'architecture du code pour comprendre son fonctionnement générale, puis on s'intéressera plus en détails à la matrice d'équilibre et à son inversion. On présentera ensuite les entrées et sorties du code.

3.1 Architecture du code

Le code est axé autour d'une fonction centrale *main*. Lorsqu'on lance le fichier contenant cette fonction, on donne en argument un fichier *.txt* qui contient les informations du système à résoudre. La fonction va ensuite appeler une après l'autre les fonctions nécessaires au bon fonctionnement du programme. Ces fonctions sont dans l'ordre :

1. `lecture_data` : Cette fonction lit le fichier *.txt* d'entrée et retourne les informations du système à résoudre.
2. `resolution` : Cette fonction prend les informations du système et retourne le vecteur de solution.
3. `print_resultats` : Cette fonction présente les résultats obtenus.

Par soucis de clarté, chaque fonction appelle elle même d'autres fonctions auxiliaires qui ont chacune leur propre rôle. On parle du fichier *.txt* d'entrée et des fonctions `lecture_data` et `print_resultats` plus en détails dans la partie

3.3. La fonction `resolution` contenant le solveur en lui-même, on parle de sa structure ici, puis de la stratégie de construction et d'inversion de la matrice d'équilibre dans la partie 3.2. La figure 4 montre un schéma synthétisant les étapes de la foncion `main` et détaille la fonction `resolution`.

Les informations du système à résoudre sont enregistrées dans deux listes d'objets : une liste d'objet de type `Noeud` et une liste d'objet de type `Barre`. Pour chaque nœud du système, on crée un objet de type `Noeud` qui contient son indice, sa position, son déplacement, la force extérieure appliquée sur lui et le type d'attache. La position, le déplacement et la force extérieure sont des vecteurs. Initialement, le déplacement de chaque `Noeud` est un vecteur nul. Pour le type d'attache, on a deux variable booléennes nommées `ux` et `uy`. Pour un nœud fixe, les deux variables sont `True`. Pour un nœud libre, les deux variables sont `False`. Pour un nœud glissant en $\vec{u}_x$, `ux = False` et `uy = True` et inversement pour un nœud glissant en $\vec{u}_y$. Pour chaque barre du système, on crée un objet de type `Barre` qui contient son indice, ses deux extrémités, sa longueur, son élongation et sa rigidité axiale. Les deux extrémités sont des objets de types `Noeud`, sa longueur est calculée à partir des positions des extrémités et l'élongation est initialement nulle.

Avec ces deux listes de nœuds et de barres, on appelle la fonction `resolution` qui va retourner les inconnues du système (les tensions dans les barres et les forces de réactions aux supports). Cette fonction procède par étapes, et va appeler plusieurs fonctions auxiliaires :

1. Calcul du nombre d'équations et d'inconnues du système. Le nombre d'équations est le double du nombre de nœuds ($2 \times n$). Pour calculer le nombre d'inconnues, on prend le nombre de barres b puis on passe en revue chaque nœud pour ajouter 0, 1 ou 2 selon types d'attaches pour obtenir le nombre d'actions de liaisons a comme dans (6). Ces informations permettent de vérifier la condition (7). Si cette condition n'est pas vérifié, le code retourne une erreur et s'arrête.
2. Appel de la fonction auxiliaire `_constru_matrice` qui va construire et retourner la matrice d'équilibre $\mathbf{K}$.
3. Appel de la fonction auxiliaire `_inversion_matrice` qui va construire l'inverse de la matrice d'équilibre $\mathbf{K}^{-1}$ et la retourner.
4. Création du vecteur des forces extérieures $\vec{F}$ en répertoriant les forces enregistrées dans les objets de type `Noeud`.
5. Calcul du vecteur des inconnues par multiplication matricielle comme dans (9).
6. Appel de la fonction auxiliaire `_deformation` qui va calculer les déformations de chaque nœud comme dans (15).

Cette fonction calcule aussi le résidu de la solution et l'affiche sur le terminal. Le résidu permet de vérifier que l'erreur de la solution est faible. Il est défini pour un ensemble de force dont au moins une est non-nulle et est calculé comme suit :

$$r = \frac{|\mathbf{K} \times \vec{S} - \vec{F}|}{|F|} \tag{24}$$

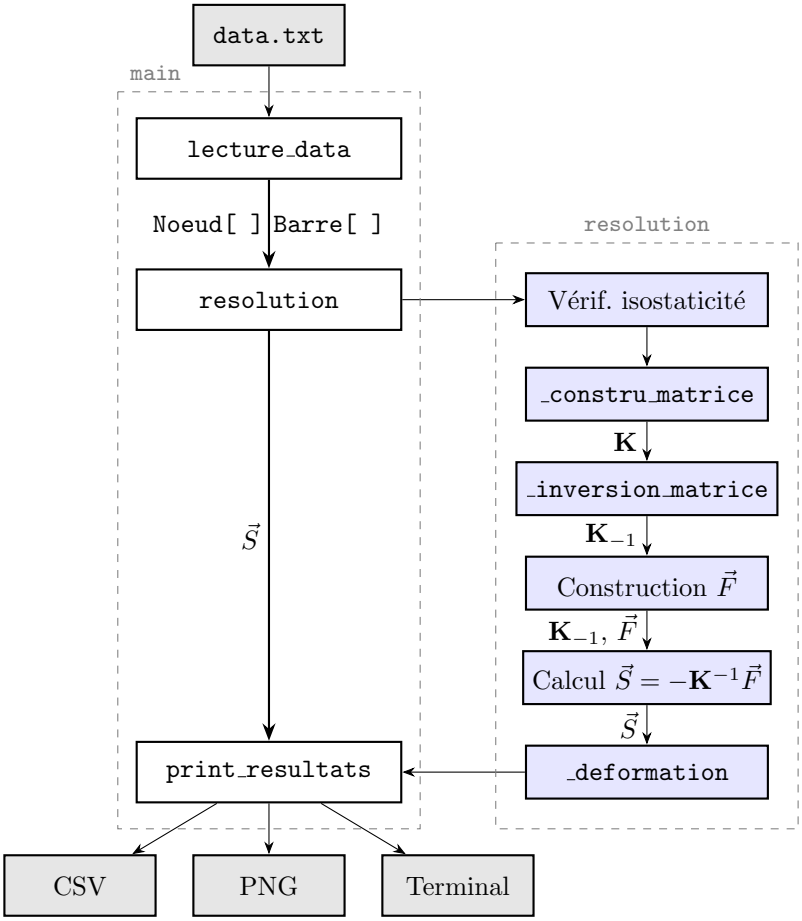


FIGURE 4 – Architecture du programme

La fonction `resolution` retourne alors le vecteur des inconnues $\vec{S}$. Le code va ensuite rendre ces informations lisibles par l'utilisateur par plusieurs moyens, comme mentionné dans la partie 3.3.

3.2 Matrice d'équilibre

3.2.1 Construction

Pour construire la matrice d'équilibre $\mathbf{K}$ on commence par initialiser une matrice carrée de termes nuls à l'aide de la librairie `numpy`. Cette matrice a pour dimensions le double du nombre de nœuds, qu'on peut récupérer simplement en prenant la longueur de la liste d'objets de type `Noeud`. On va ensuite faire une boucle sur les nœuds `j`, chaque nœud contribuant à deux lignes `2j` et `2j+1`. Chaque paire de ligne correspond aux projections sur $\vec{u}_x$ et $\vec{u}_y$ de l'équation

d'équilibre d'un nœud `j`.

Pour remplir ces lignes, on passe en revue chaque colonne `i` en commençant par les b premières qui correspondent aux tensions dans chaque barre. Si la barre b_i correspondant à la colonne `i` est connectée au nœud `j`, on va assigner les bons coefficients aux cases `K[2j][i]` et `K[2j+1][i]` comme expliqué dans la partie 2.2. Ces coefficients sont les sinus et cosinus qui encodent les relations géométriques du système. On doit prendre soin de bien orienter ces sinus et cosinus, c'est-à-dire de les définir positifs ou négatifs selon si le nœud `j` est la première ou la seconde extrémité de la barre b_i.

On passe ensuite sur les a colonnes correspondant aux réactions de liaisons. En fonction des valeurs des variables booléennes `ux` et `uy` du nœud `j`, on va modifier ou non les cases adéquates. On crée une variable `idx` pour avoir mémoire de la colonne concernée, initialement `idx=b`. Si le nœud `j` est sans attaches, on passe aux deux lignes suivantes (au nœud `j+1`). Si le nœud `j` est fixe, on assigne `1` aux cases `K[2j][i]` et `K[2j+1][i+1]` et on incrémente deux fois `idx` : `idx=idx+2`. Si le nœud `j` est glissant en $\vec{u}_y$, on assigne `1` à la case `K[2j][i]` et on incrémente `idx` : `idx=idx+1`. Idem pour un nœud glissant en $\vec{u}_x$, avec `K[2j+1][i]`.

Eventuellement rajouter exemple comme dans 2.5. On donne le pseudo-code de cette fonction en annexe A.

3.2.2 Inversion et singularité

Pour inverser la matrice d'équilibre, on utilise l'algorithme d'élimination de Gauss-Jordan. Cet algorithme transforme la matrice $\mathbf{K}$ en matrice identité en lui appliquant des opérations élémentaires sur les lignes, tout en appliquant les mêmes opérations à une matrice identité initialement accolée, qui devient alors $\mathbf{K}^{-1}$. L'encyclopédie en ligne Wikipédia propose un article clair sur le sujet de l'élimination de Gauss-Jordan. Pour éviter de surcharger ce document, on renvoie à cet article [5] et on donne le pseudo-code de cette fonction en annexe B.

L'algorithme utilise une stratégie de *pivot partiel* : plutôt que de diviser par le coefficient diagonal courant, on cherche d'abord le plus grand coefficient de la colonne et on réorganise les lignes en conséquence. Cela évite de diviser par des valeurs très petites, ce qui pourrait introduire des erreurs numériques importantes.

Si au cours de l'exécution de l'algorithme tous les coefficients d'une colonne sont proches de zéro (inférieurs à 10^{-7} ici), la matrice est considérée comme singulière et le programme s'arrête avec un message d'erreur. Concrètement, une matrice $\mathbf{K}$ singulière signifie que le système n'a pas de solution unique, la structure peut bouger sans se déformer : c'est un *mécanisme*.

Cela peut arriver dans deux cas. Le premier est simple : la structure n'a pas assez d'appuis et peut se déplacer librement. Le second est plus subtil : même si la condition d'isostaticité (7) est vérifiée, certaines géométries particulières posent problème. Par exemple, si toutes les barres sont parallèles, elles sont incapables d'équilibrer une force perpendiculaire à leur direction. La condition

(7) est donc nécessaire mais pas suffisante : un treillis peut vérifier $2n = a + b$ et être quand même un mécanisme. On donne pour exemple le mécanisme de la figure 5. Ce mécanisme est composé de trois nœuds dont deux fixes et un libre et deux barres. La barre b_2 est libre de tourner autour du nœud 2. La matrice d'équilibre du système $\mathbf{K}_{mecanisme}$ est une matrice singulière, comme on peut le voir ici :

$$\mathbf{K}_{mecanisme} = \begin{pmatrix} cos(\theta_{1,2}) & 0 & 1 & 0 & 0 & 0 \\ sin(\theta_{1,2}) & 0 & 0 & 1 & 0 & 0 \\ cos(\theta_{2,1}) & cos(\theta_{2,3}) & 0 & 0 & 1 & 0 \\ sin(\theta_{2,1}) & sin(\theta_{2,3}) & 0 & 0 & 0 & 1 \\ 0 & cos(\theta_{3,2}) & 0 & 0 & 0 & 0 \\ 0 & sin(\theta_{3,2}) & 0 & 0 & 0 & 0 \end{pmatrix} \tag{25}$$

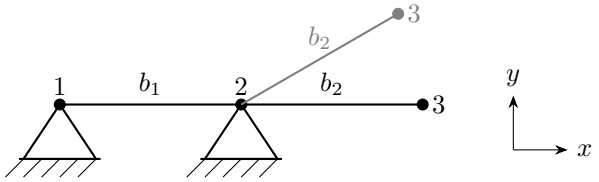


FIGURE 5 – Mécanisme isostatique

3.3 Entrée et sorties

3.3.1 Entrée

Les informations du système sont données au programme sous forme de texte dans un fichier `.txt` et le code rend les solutions du système sous forme de fichiers `.png` et `.csv`. Tout ces fichiers sont une forme d'interface entre l'utilisateur et la machine. Leurs rôles est de communiquer les informations du système à résoudre et ses solutions de manière claire et sans ambiguïté.

Le texte d'entrée est séparé en trois parties qui donnent les informations sur les paramètres du code, sur les nœuds et sur les barres. Chaque partie commence par une ligne indiquant la partie en question avec un mot : "`Parametres:`", "`Noeuds:`" ou "`Barres:`". La partie `Parametres` est constituée d'une ligne elle même constituée de cinq termes. Ces termes sont dans l'ordre une chaîne de caractère `out_dir`, trois variables booléennes `save_csv`, `save_png` et `show_png` et un nombre `ampli`. La chaîne de caractère `out_dir` donne le nom du dossier dans lequel seront enregistrés les différents fichiers de sortie du code. Les variables booléennes `save_csv`, `save_png` et `show_png` permettent de commander au code d'enregistrer et de montrer ou non les fichiers `.png` et `.csv`. Ces variables booléenne peuvent être exprimé comme `True/False` ou `1/0`. La partie `Noeuds` est organisée en lignes, chaque ligne représentant un nœud. Ces lignes sont constituées de cinq termes (quatre nombres et un caractère) qui sont dans

l'ordre les coordonnées x et y du nœud, les composantes F_x et F_y de la force extérieure appliquée sur lui et un caractère indiquant son type d'attache. Le type d'attache peut être exprimé comme `f`, `x`, `y` ou `l`, respectivement pour un nœud fixe, fixe en $\vec{u}_x$, fixe en $\vec{u}_y$ ou libre. La partie `Barres` est organisé en lignes, chaque ligne représentant une barre. Ces lignes sont constituées de trois termes (trois nombres) qui sont dans l'ordre les indices des nœuds/extrémités de la barre et sa rigidité axiale EA. L'annexe C donne un fichier texte type et un exemple pour le système représenté par la figure 6.

Si le fichier d'entrée n'est pas exactement dans le format spécifié, le code a quelques mécanismes pour détecter les erreurs. Par exemple si le type d'attache de nœud n'est pas un caractère entre `f`, `x`, `y` ou `l`, on inscrit le nœud comme étant libre. On peut aussi arrêter le code si une ligne ne contient pas le bon nombre de termes ou si les termes ne sont pas du bon type de variable. Par exemple pour une ligne dans la partie `Noeuds` sans le type d'attache, le code renverra une erreur : "`Ligne noeud invalide. Attendu: 'x y fx fy liaison'.`"

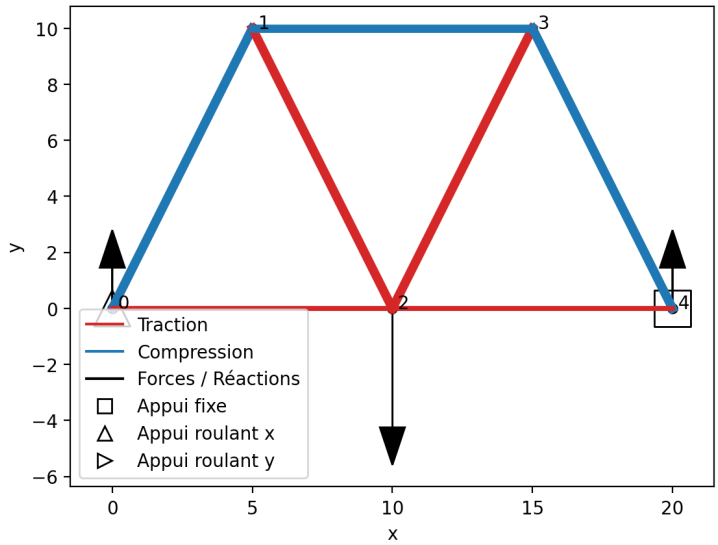


FIGURE 6 – Résultat du code pour un treillis avec une force extérieure

3.3.2 Sorties

Le code a trois manières de présenter ses résultats : il écrit quelques informations importantes sur le terminal, affiche deux images `.png` et enregistre deux fichier `.csv` et deux fichiers `.png`.

Le texte sur le terminal permet à l'utilisateur d'obtenir rapidement les informations générales comme le résidu de la solution, les tensions de compression et de traction maximales ou les réactions aux supports.. Comme précisé plus

tôt, le texte du terminal permet aussi de donner des informations sur les erreurs si le code en a rencontrées. Ce texte sur le terminal permet d'avoir un retour directement après l'éxecution du code.

Les fichiers `.csv` présentent tous les résultats numériques dans un fichier de texte avec les valeurs séparées par des virgules. C'est un format universel, il peut être lu sur Excel, LibreOffice ou beaucoup d'autre outil. Il donne toutes les valeurs numériques et permets de les conserver. De plus il permet de séparer la phase de calcul et la phase de visualisation/analyse : on peut l'utiliser pour visualiser et analyser les résultats sans relancer le calcul. Pour un code plus complexe, ou avec des systèmes plus grands que ceux présentés en exemple, les calculs peuvent être chronophages et cette séparation entre calculs et post-processing peut être utile.

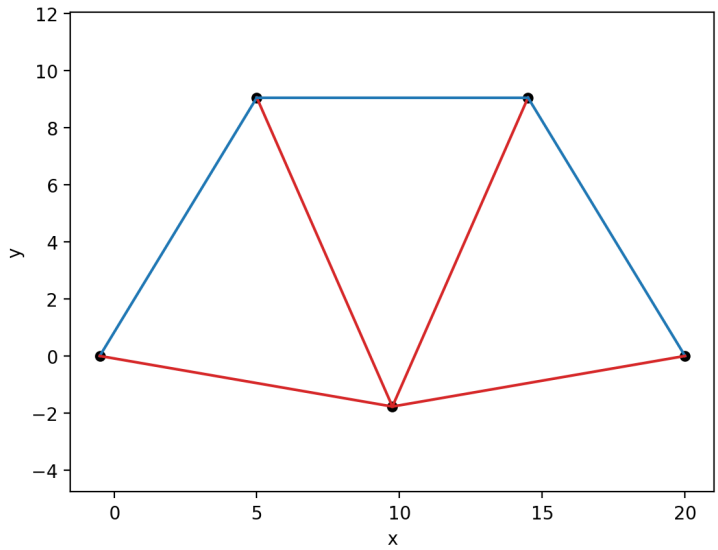


FIGURE 7 – Résultat du code pour un treillis avec déformation amplifiée

Le code produit deux images `.png`. Une première image présente un schéma du treillis avec des barres plus ou moins larges selon la norme de leurs tensions, et avec des couleurs pour repérer les barres en compressions ou en tractions. Cette image montre aussi les forces extérieures appliquées au noeuds et les réactions aux supports. Une seconde image présente un schéma du treillis déformé avec une amplification plus ou moins importante selon le parametre choisi en entrée. Ces deux images servent à visualiser facilement la solution du système mais ne présente pas les valeurs numériques. Les figures 6 et 7 montrent ces deux images pour l'exemple `exemple1.txt` disponible sur GitHub [1] et dont le texte d'entrée est présenté dans l'annexe C.

4 Conclusion

Ce projet a permis de construire un solveur pour les treillis isostatiques en deux dimensions. À partir d'un fichier de description du système, le programme calcule les tensions dans les barres, les réactions aux appuis et les déplacements nodaux, et restitue ces résultats sous plusieurs formes.

Pour valider ce code, on a pris pour exemple des treillis simple pour comparer les resultats du code au calcul fait à la main. Dans un code destiné à un usage professionnel, la validation du solveur demanderait des étapes plus rigoureuses. Il y a généralement deux niveaux de validation. Le premier est la *vérification* : on s'assure que le code résout correctement les équations qu'il est censé résoudre, par exemple en comparant les résultats à des solutions analytiques connues. Le second est la *validation* : on compare les résultats à des mesures expérimentales ou à des résultats issus de logiciels commerciaux reconnus comme par exemple ANSYS. Ces deux étapes sont indispensables avant de pouvoir utiliser un solveur dans un contexte industriel.

Ce solveur repose sur plusieurs hypothèses qui en limitent le champ d'application. Il ne traite que les systèmes isostatiques en deux dimensions, avec des barres à comportement élastique linéaire et des petits déplacements. Les systèmes hyperstatiques, les problèmes en trois dimensions, les comportements non linéaires ou les grands déplacements ne sont pas pris en charge. Par ailleurs, l'inversion directe de la matrice d'équilibre, bien que fonctionnelle pour de petits systèmes, devient rapidement coûteuse en temps de calcul pour des structures avec un grand nombre de nœuds.

Ce travail constitue cependant une première étape vers un code de méthode des éléments finis (MEF). La MEF utilise les déplacements nodaux comme inconnues principales, ce qui permet de traiter les systèmes hyperstatiques. Elle repose sur la construction d'une matrice de rigidité globale assemblée à partir des matrices de rigidité élémentaires de chaque barre, et se résout par le même type de système linéaire $\mathbf{K}\vec{u} = \vec{F}$, avec $\vec{u}$ le vecteur des déplacements nodaux. Les notions de maillage, d'assemblage et de conditions aux limites abordées seront des sujets à explorer pour pouvoir continuer vers un code de MEF.

Annexes

A Pseudo-code de la fonction _constru_matrice

```
Entrée  : liste de Noeud, liste de Barre
Sortie  : matrice K de dimensions (2n x 2n)

ndof <- 2 * nombre de noeuds
K    <- matrice nulle de dimensions (ndof x ndof)
idx  <- nombre de barres

Pour j de 0 à ndof-1 par pas de 2 :
    n <- j / 2                          // indice du noeud courant

    Pour i de 0 à nombre de barres - 1 :
        Si barre[i].n1 == noeud[n] :    // noeud est la 1ère extrémité
            K[j  ][i] <- (barre[i].n2.x - noeud[n].x) / barre[i].longueur
            K[j+1][i] <- (barre[i].n2.y - noeud[n].y) / barre[i].longueur
        Sinon si barre[i].n2 == noeud[n] :  // noeud est la 2ème extrémité
            K[j  ][i] <- (barre[i].n1.x - noeud[n].x) / barre[i].longueur
            K[j+1][i] <- (barre[i].n1.y - noeud[n].y) / barre[i].longueur

    // Colonnes des réactions de liaison
    Si noeud[n].ux == True et noeud[n].uy == False :   // glissant en uy
        K[j][idx] <- 1
        idx <- idx + 1
    Sinon si noeud[n].ux == False et noeud[n].uy == True :  // glissant en ux
        K[j+1][idx] <- 1
        idx <- idx + 1
    Sinon si noeud[n].ux == True et noeud[n].uy == True :   // fixe
        K[j  ][idx  ] <- 1
        K[j+1][idx+1] <- 1
        idx <- idx + 2

Retourner K
```

B Pseudo-code de la fonction _inversion_matrice

```
Entrée  : matrice carrée A de dimensions (n x n)
Sortie  : matrice inverse de dimensions (n x n)

seuil <- 1e-7

// Construction de la matrice augmentée [A | I]
Aug <- matrice nulle de dimensions (n x 2n)
Pour i de 0 à n-1 :
    Pour j de 0 à n-1 :
        Aug[i][j] <- A[i][j]
    Aug[i][n+i] <- 1              // accoler la matrice identité

// Élimination de Gauss-Jordan
Pour col de 0 à n-1 :

    // Recherche du pivot partiel
    idxPivot <- col
    max      <- |Aug[col][col]|
    Pour row de col+1 à n-1 :
        Si |Aug[row][col]| > max :
            max      <- |Aug[row][col]|
            idxPivot <- row

    // Vérification de la singularité
    Si max < seuil :
        Lever une erreur : "Matrice singulière, système irrésoluble"

    // Permutation de lignes si nécessaire
    Si idxPivot != col :
        Échanger Aug[col] et Aug[idxPivot]

    // Normalisation de la ligne pivot
    pivot <- Aug[col][col]
    Pour j de 0 à 2n-1 :
        Aug[col][j] <- Aug[col][j] / pivot

    // Élimination sur toutes les autres lignes
    Pour row de 0 à n-1 :
        Si row == col : continuer
        facteur <- Aug[row][col]
        Si |facteur| < seuil : continuer
        Pour j de 0 à 2n-1 :
            Aug[row][j] <- Aug[row][j] - facteur * Aug[col][j]
```

```
// Extraction de l'inverse (partie droite de Aug)
inverse <- matrice nulle de dimensions (n x n)
Pour i de 0 à n-1 :
    Pour j de 0 à n-1 :
        inverse[i][j] <- Aug[i][n+j]

Retourner inverse
```

C Exemple de texte d'entrée du code

Le premier texte est un texte type pour un treillis avec trois nœuds et trois barres. On note `str` pour une chaîne de caractères, `bool` pour une variable booléenne, `float` pour un nombre et `char` pour un caractère.

```
Parametres:
str bool bool bool float

Noeuds:
float float float float char
float float float float char
float float float float char

Barres:
float float float
float float float
float float float
```

Le second texte est un exemple pour un treillis de cinq nœuds et sept barres avec une force extérieures. Ce texte est le contenu du fichier d'entrée de l'exemple `exemple1.txt` disponible sur GitHub [1].

```
Parametres:
outputs FALSE TRUE TRUE 50

Noeuds:
0 0 0 0 y
5 10 0 0 l
10 0 0 -20 l
15 10 0 0 l
20 0 0 0 f

Barres:
0 1 10000
1 2 10000
2 3 10000
3 4 10000
0 2 10000
2 4 10000
1 3 10000
```

Références

[1] Max Billette. *MecaStruct.* GitHub, 2026.
https://github.com/Max7509/MecaStruct

[2] Patrice Cartraud. *Mécanique des structures. Étude des poutres.* École Centrale de Nantes, France, 2011, pp. 67. cel-00451733v2

[3] A. Delaplace, F. Gatuingt, F. Ragueneau. *Mécanique des structures – Résistance des matériaux.* Dunod, Paris, 2008.

[4] Philippe Malbos. *Analyse matricielle et algèbre linéaire appliquée.* Université Claude Bernard Lyon 1.

[5] Wikipédia. *Élimination de Gauss-Jordan.*
https://fr.wikipedia.org/wiki/limination_de_Gauss-Jordan.